

5.1 Itens: types, helpers e regras de cálculo — status de entrega

Bloco 5.1 Itens: types, helpers e regras de cálculo · Sprint 5 — Itens (o coração do sistema) · Entregue em 2026-09-21, ajustado em 2026-09-23 · Branch: `feature-dev-breno-automacao` · Ambiente: gc-dev (`gzbmhgnpoehornidmegg`)

Resumo

Bloco concluído. `src/lib/itens.ts` com 25 exports e 85 casos de teste unitário (baseline da sprint 4 era 51). A camada de dados ganhou a leitura da view `itens_com_status`, que o tipo declarava sem prova.

É também o ponto de contato com a trilha de automação: a Fase 6 da migração para Telegram grava itens vindos do OCR e reusa estes helpers, em vez de reescrever a regra em JavaScript de nó n8n.

1. As quatro atividades do bloco

ATIVIDADE DO CLICKUP	ESTADO
<code>Item</code> / <code>ItemInsert</code> / <code>ItemUpdate</code> e <code>ItemComStatus</code> em <code>src/lib/types.ts</code>	já existiam; estreitados , e a view passou a ser lida
<code>src/lib/itens.ts</code> com helpers de unidade (QTD/M2) e formatação de dimensões	criado , 25 exports
Documentar que <code>valor_total</code> e <code>area_m2</code> são GENERATED e NÃO entram no payload	documentado e imposto por tipo
Helper de numeração automática: próximo número dentro da proposta ou contrato	criado

1.1 Tipos — o que faltava era estreitar

`Item`, `ItemInsert`, `ItemUpdate` e `ItemComStatus` já estavam em `src/lib/types.ts`. O que não existia:

- `Unidade = 'QTD' | 'M2'`, e o narrowing de `Item.unidade` e `ItemComStatus.unidade`, que vinham como `string` do gerador — mesmo tratamento que `OrcamentoStatus` e `PropostaStatus` já recebiam. Registrado no tipo que **ML foi removido de propósito** na `20260511151924_revisao_schema.sql` §8, que migrou os registros existentes para `QTD`.
- `ItemPayload` e `ItemUpdatePayload` — ver 1.2.

1.2 As colunas GENERATED: documentadas e impossíveis

A atividade pedia documentar. Documentar não bastava — o erro passa por todas as camadas baratas e só aparece no banco:

```
valor_total generated always as (valor_unit * quantidade) stored
area_m2      generated always as (largura * altura * quantidade) stored
```

O gerador de types do Supabase não sabe que são geradas. Conferido no `src/lib/supabase/types.ts`: `items.Insert` traz `valor_total?: number | null` e `area_m2?: number | null` como qualquer outra coluna. Ou seja, `valor_total: 200` compila, passa no `npm test`, e só estoura no Postgres com `cannot insert a non-DEFAULT value into column "valor_total"`.

Por isso `ItemPayload` faz `Omit` das duas. **Prova de que funciona** — arquivo temporário com os dois erros clássicos, compilado e depois removido:

```
error TS2353: Object literal may only specify known properties,
      and 'valor_total' does not exist in type 'ItemPayload'.
error TS2322: Type '"ML"' is not assignable to type 'Unidade | null | undefined'.
```

Os dois erros saíram da camada `dados / escrita` e foram para a `estatico`, que roda primeiro e custa 8s.

Segunda barreira, para payload que não passa pelo TypeScript (corpo da rota de ingestão da Fase 6, linha de planilha do bloco 5.4): `COLUNAS_GERADAS_ITEM` e `limparColunasGeradas()`.

1.3 `src/lib/itens.ts` — 25 exports

Helper puro, sem `use client` e sem import de React, como manda o `CLAUDE.md`.

GRUPO	O QUE TEM
Colunas geradas	<code>COLUNAS_GERADAS_ITEM</code> , <code>limparColunasGeradas</code>
Unidade	<code>UNIDADES</code> , <code>UNIDADE_LABELS</code> , <code>UNIDADE_SUFIKOS</code> , <code>isUnidade</code> , <code>formatUnidade</code> , <code>formatUnidadeSufixo</code> , <code>normalizarUnidade</code>
Dimensões	<code>formatDimensao</code> , <code>formatDimensoes</code> , <code>formatArea</code>
Espelhos das geradas	<code>areaDoItem</code> , <code>valorTotalDoItem</code>
Somas	<code>somaItens</code> , <code>somaQuantidade</code> , <code>somaArea</code> , <code>totaisDosItens</code>
Numeração	<code>proximoNumeroItem</code> , <code>numeroItemDoDocumento</code>
Valor unitário ausente	<code>resolverValorUnit</code> , <code>notaDeInferencia</code> , <code>acrescentarObservacao</code>

GRUPO	O QUE TEM
Vínculo	<code>vinculoValido</code>

Quatro decisões que valem explicação:

- `proximoNumeroItem` é **maior + 1**, não `length + 1`. `itens.numero` tem unique parcial por pai (`idx_itens_numero_proposta`, `idx_itens_numero_contrato`). Com um item excluído no meio, `length + 1` colidiria com um número que ainda existe. Tem teste para esse caso.
- `areaDoItem` e `valorTotalDoItem` devolvem `null`, não zero, quando falta um fator. Zero é um valor plausível que a tela exibe sem reclamar; `null` vira "-" e a pessoa vê que falta dado.
- `normalizarUnidade` nunca devolve `null`. Item sem unidade legível é contado por peça, que é o default da coluna. O sinalizador `reconhecida: false` diz que houve chute, para quem quiser registrar o original em `observacao`.
- `totaisDosItens` existe para o rodapé não repetir `reduce` por coluna. A primeira versão do 5.2 fez isso, e a soma passou a viver em dois lugares — um testado e outro não. O agrupador fecha essa porta.

1.4 `resolverValorUnit` — escrito aqui porque a automação precisa

Não estava na lista do bloco, mas é helper puro de item, e o plano de migração define `src/lib/` como o ponto de contato entre as duas frentes, com a regra "quem chegar primeiro escreve".

O caso: o PDF de proposta real traz a maioria dos itens **só com "Valor total"**, sem unitário. A intuição é gravar `valor_unit: null` e preservar o total — e isso **não funciona**, porque `valor_total` é gerado a partir do unitário. Gravar o unitário nulo não preserva o total: apaga o valor do item.

Então `resolverValorUnit` deriva `valor_total / quantidade`, marca `inferido: true` para o chamador registrar o rastro em `observacao`, e **recusa** os dois casos sem saída — quantidade nula ou zero, e item sem nenhum dos dois valores. Nesses, quem chama manda o documento para `REVISAO_HUMANA`; nunca grava zero.

2. Conferência contra o documento real — e uma correção no plano

Em vez de confiar nos casos que eu havia imaginado, extraí `keen-mendel/relatorios/documentos-fonte/PROP. COM. - EF _ ADITIVOS_15 12 2025.pdf` (proposta `EB-25-08-0048`) com `pdftotext - layout`. Três achados:

1. **São 16 itens, não 12.** O plano de migração afirmava 12 em três lugares. Corrigido lá, com o método de conferência registrado.
2. **11 dos 16 itens não têm valor unitário, e todos os 16 têm valor total.** É a confirmação mais forte possível da regra de 1.4: gravar `valor_unit` nulo apagaria **mais de R\$ 115 mil** de um documento de ~R\$ 121 mil. Há teste somando exatamente isso.
3. **A numeração é inteiro simples de 1 a 16** — não há `1.1` nem `1A` neste documento. Ver pendência 2 da Seção 4.

Quantidade vem com zero à esquerda (01 , 02 , 13), e o item 9 (13 × 120,00 = 1.560,00) é o único com quantidade > 2 e unitário presente — serve de controle de que a conta do banco bate com o papel.

3. Números reais do `npm run validar`

Rodado em 2026-09-22 contra gc-dev, comparado com a baseline de 2026-09-10.

CAMADA	BASELINE	AGORA	
estático	limpo	<code>tsc --noEmit</code> limpo, <code>✓ No ESLint warnings or errors</code> (8s)	ok
unitário	51 casos	85 casos, 0 falhas (1s)	+34
build	31 rotas	31 rotas, iguais à baseline	ok
runtime	41/41 rotas	43/43 rotas ok	+2 (5.2)
dados	12/12 checagens	15/15 checagens ok	+1 aqui, +2 no 5.2
escrita	38/38 passos	61/61 passos ok	5.2
navegador	24/24 passos	34/34 passos ok	5.2

Os 34 casos unitários novos cobrem: as duas colunas geradas e a limpeza de payload; o mapa de unidade (incluindo `ML` → `QTD` e o desconhecido); formatação de dimensão e área; os espelhos das colunas geradas devolvendo `null` em vez de zero; as quatro somas; numeração com buraco no meio; `numero` não inteiro virando `null` com o original preservado; a inferência de unitário e os dois casos que ela recusa; o XOR do vínculo; e os quatro casos extraídos do PDF real.

3.1 A checagem de dados deste bloco

[5.1] `itens_com_status`: view de item com execução (`security_invoker`) — **15 registros**.

`ItemComStatus` foi estreitado aqui, mas a view nunca havia sido **lida**: o tipo existia sem prova de casar com o que o PostgreSQL devolve. A checagem confere que os `coalesce` do LEFT JOIN aplicam — item sem execução tem de vir `'pendente'`, não `null` — e que `unidade` respeita o CHECK. Se alguém reintroduzir `ML` por SQL direto, o narrowing de `Unidade` vira mentira e esta checagem acusa.

Fechamento da sprint 5 — o que mudou neste bloco

Revisão de 2026-09-23: os seis documentos da sprint foram relidos, e cada pendência nominal foi corrigida ou justificada. Números do plano inteiro no fim da revisão:

CAMADA	BASELINE (SPRINT 4)	FIM DA SPRINT 5
estático	limpo	<code>tsc --noEmit</code> limpo, <code>✓ No ESLint warnings or errors</code>

CAMADA	BASELINE (SPRINT 4)	FIM DA SPRINT 5
unitário	51 casos	140 casos, 0 falhas
build	31 rotas	32 rotas, iguais à baseline
runtime	41/41 rotas	47/47 rotas ok
dados	12/12 checagens	18/18 checagens ok
escrita	38/38 passos	101/101 passos ok
navegador	24/24 passos	50/50 passos ok, console limpo

Rodado em 2026-09-23 contra gc-dev. `migration list` : 17/17 alinhadas. Depois da rodada, conferido por query: 0 propostas de teste, 0 contratos de teste, 0 itens órfãos, 0 arquivos de item no bucket.

O que a revisão acrescentou aos helpers deste bloco:

- `CAMPOS_EDITAVEIS_ITEM` e `camposEditaveisItem()` — a whitelist de escrita que fechou o furo de segurança das actions do 5.2 (ver o documento do 5.2, 2.8). 5 casos de teste.
- `divergenciaDeValor()` — a comparação valor × soma do 5.6, com o caso da soma abaixo do desconto. 5 casos.
- Dois helpers novos em `src/lib/`, no mesmo espírito deste bloco — puros, testados por `node --test : itens-planilha.ts` (a leitura da planilha do 5.4, 13 casos) e o teste de `files.ts`, que lê os tipos do bucket direto da migration (3 casos).

Os 44 casos de `src/lib/itens.test.ts` são deste bloco e dos ajustes que ele recebeu.

4. O que NÃO foi validado — pendências nominais

1. Cinco helpers seguem sem consumidor em produção: `normalizarUnidade`, `numeroItemDoDocumento`, `resolverValorUnit`, `notaDeInferencia` e `acrescentarObservacao`. São da Fase 6 da automação — quem os chama é a rota de ingestão, que ainda não existe. Estão testados e esperando; um ponto de uso artificial só para zerar a lista seria pior.
2. `numero` não-inteiro nunca foi visto num documento real. O PDF de referência numera de 1 a 16 em inteiros simples; o caminho `1.1 / 1A` é coberto só por caso hipotético. Depende de aparecer um documento assim — não é algo que se resolva escrevendo código.
3. `normalizarUnidade` não foi confrontada com o vocabulário real do OCR. Mesmo motivo: só a Fase 5 da automação produz esse vocabulário.

5. Próximo passo

Os helpers estão prontos para os blocos seguintes da sprint (5.3 e 5.4) e para a Fase 6 da automação, que consome `normalizarUnidade`, `resolverValorUnit`, `numeroItemDoDocumento`, `limparColunasGeradas` e `vinculoValido`.